



Security Audit Report

ZIG Chain v5

v2.0

October 5, 2026

Table of Contents

[Table of Contents](#)

[License](#)

[Disclaimer](#)

[Introduction](#)

[Purpose of This Report](#)

[Codebase Submitted for the Audit](#)

[Methodology](#)

[Functionality Overview](#)

[How to Read This Report](#)

[Code Quality Criteria](#)

[Summary of Findings](#)

[Detailed Findings](#)

- [1. Depositing a trivial amount into the migration faucet or burner address ahead of the upgrade height trips the supply-conservation check and crashes every validator](#)
- [2. Chain-wide denial of service enabled by an ordinary active governance proposal that becomes undecodable at the v5 upgrade height](#)
- [3. A dust uzig deposit at a not-yet-created pool address halts the upgrade once that pool is created, because the DEX mirror reconciliation tolerates no divergence](#)
- [4. A wasm StoreCodeAuthorization grant with an attacker-chosen CodeHash can cause the authz migration's fail-closed substring scan and halt the upgrade](#)
- [5. A group proposal embedding a wire-incompatible message permanently wedges the x/group end-blocker's chain-wide tally and pruning scan after the upgrade](#)
- [6. CosmWasm contracts that track uzig amounts in their own internal storage will desync from their real balance after the v5 redenomination](#)
- [7. Near-zero DEX pool-creation and factory denom-creation fees on freshly initialized chains due to unrescaled default fee constants](#)
- [8. Node minimum gas prices are not migrated and require validator coordination](#)
- [9. DEX AddLiquidity/RemoveLiquidity response fields mislabel Base/Quote by sorted array position instead of caller-supplied role after the v5 redenomination](#)
- [10. Packet-forward-middleware forwarding of a returning uzig voucher always fails due to migrateuzig converting the intermediate balance to azig before the forward attempt](#)
- [11. Staking HistoricalInfo retains old-scale validator tokens after the upgrade](#)
- [12. An IBC destination callback executes against uzig while the receive settles in azig](#)
- [13. A long-running v5 upgrade handler cannot be aborted or forked by CometBFT consensus round timeouts, but slow validators risk downtime-jailing rather than a protocol-level guarantee of recovery](#)
- [14. The v5 binary cannot replay pre-upgrade blocks because the power reduction is set unconditionally](#)
- [15. An error-acknowledgement refund finalizes with spendable legacy uzig when the faucet is short](#)

16. Active governance proposals with embedded uzig amounts are not rescaled

17. Code quality improvements

18. Wasm authorization message filters are not redenominated

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Highend Technologies LLC to perform a security audit of the pull request 14 (chain update).

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	URL	Type	Commit
zigchain-private	https://github.com/ZIGChain/zigchain-private	Pull Request #14	Base: zigchain-private@73877afd31 Head: zigchain-private@02381442c3 Compare changes

For repositories reviewed as a Diff or Pull Request above, this audit covers only the changes introduced between the base and head references shown, and does not constitute a full review of the codebase at the head commit.

Scope	<pre>app/ keepers/ keepers_config.go keepers.go upgrades/ v4_1/ constants.go upgrades.go v5/ redenom/ redenom.go snapshot.go auth.go authz.go bank.go constants.go dex.go distribution.go errors.go escrow.go factory.go faucet.go feegrant.go gov.go mint.go ratelimit.go staking.go tokenwrapper.go upgrades.go utils.go verify.go basic_manager_overrides.go config.go</pre>
-------	---

```

export.go
genesis.go
ibc.go
setup_handlers.go
cmd/
  zigchaind/
    cmd/
      root.go
proto/
  zigchain/
    dex/
      params.proto
    factory/
      genesis.proto
      params.proto
      query.proto
x/
  dex/
    keeper/
      k_get_set_params.go
      msg_server_create_pool.go
    migrations/
      v3_migration.go
    module/
      module.go
      simulation.go
    types/
      params_legacy.go
      params.go
  factory/
    keeper/
      denom_auth.go
      identity_freeze.go
      invariants.go
      k_get_set_params.go
      k_set_denom.go
      msg_server_create_denom.go
      msg_server_set_denom_metadata.go
      query_denom_auth.go
    migrations/
      v3_migration.go
    module/
      autocli.go
      genesis.go
      module.go
      simulation.go
    types/
      errors.go
      genesis.go
      params_legacy.go
      params.go
  migrateuzig/
    discriminator.go
    doc.go
    errors.go
    expected_keepers.go

```

```
    ibc_module.go
tokenwrapper/
    keeper/
        keeper.go
    module/
        module.go
zutils/
    constants/
        denom.go
    v5migration.go
```

Fixes Verified at Commit

These are the commits at which the client's fixes were reviewed. The status of each finding in this report (Resolved, Partially Resolved or Acknowledged) reflects the code at these commits.

fixes-review: [zigchain-private@bc2d713fb3](#)

Published at: <https://github.com/ZIGChain/zigchain> @ [642d3baedf](#)

The git tree hash of the private repository at commit bc2d713fb3 is 29ce87d155b4bd785853e3712ade740aa932b5b2. The public repository at commit 642d3baedf resolves to the same tree hash, confirming that both repositories contain identical source trees. This can be independently verified by running `git rev-parse <commit>^{tree}` on either repository.

Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

Functionality Overview

ZIGChain is a Cosmos SDK and CometBFT-based blockchain. The repository includes the core application setup, module configurations, and chain upgrade logic. Key custom modules include `factory`, which handles token denom creation, metadata, authorization, and identity freezing; `dex`, which manages liquidity pool creation and associated parameters; `tokenwrapper`, which wraps token functionality; and `migrateuzig`, which handles IBC-based migration logic with discriminator support.

The codebase follows standard Cosmos SDK module patterns, with each module containing keeper logic, message servers, query handlers, genesis management, and migration paths. The `app/upgrade` directory defines versioned upgrade handlers, with the v5 upgrade being notably broad, covering redenomination, staking, governance, distribution, fee grant, rate limiting, escrow, faucet, and authz configurations. A `zutils` package provides shared constants and migration utilities used across modules.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium-High	19-step atomic upgrade handler touching multiple modules with hand-decode wire-format bridges and cross-module ordering dependencies; 3 Critical findings trace directly to unhandled cross-module interaction edge cases
Code readability and clarity	Medium-High	Extensive inline rationale comments
Level of documentation	Medium	—
Test coverage	Medium-High	Test coverage reported by go test is 71.90%.

Summary of Findings

No	Description	Severity	Status
1	Depositing a trivial amount into the migration faucet or burner address ahead of the upgrade height trips the supply-conservation check and crashes every validator	Critical	Resolved
2	Chain-wide denial of service enabled by an ordinary active governance proposal that becomes undecodable at the v5 upgrade height	Critical	Resolved
3	A dust uzig deposit at a not-yet-created pool address halts the upgrade once that pool is created, because the DEX mirror reconciliation tolerates no divergence	Critical	Resolved
4	A wasm StoreCodeAuthorization grant with an attacker-chosen CodeHash can cause the authz migration's fail-closed substring scan and halt the upgrade	Critical	Resolved
5	A group proposal embedding a wire-incompatible message permanently wedges the x/group end-blocker's chain-wide tally and pruning scan after the upgrade	Major	Resolved
6	CosmWasm contracts that track uzig amounts in their own internal storage will desync from their real balance after the v5 redenomination	Minor	Acknowledged
7	Near-zero DEX pool-creation and factory denomination fees on freshly initialized chains due to unrescaled default fee constants	Minor	Resolved
8	Node minimum gas prices are not migrated and require validator coordination	Minor	Acknowledged
9	DEX AddLiquidity/RemoveLiquidity response fields mislabel Base/Quote by sorted array	Minor	Acknowledged

No	Description	Severity	Status
	position instead of caller-supplied role after the v5 redenomination		
10	Packet-forward-middleware forwarding of a returning uzig voucher always fails due to migrateuzig converting the intermediate balance to azig before the forward attempt	Minor	Acknowledged
11	Staking HistoricalInfo retains old-scale validator tokens after the upgrade	Informational	Acknowledged
12	An IBC destination callback executes against uzig while the receive settles in azig	Informational	Acknowledged
13	A long-running v5 upgrade handler cannot be aborted or forked by CometBFT consensus round timeouts, but slow validators risk downtime-jailing rather than a protocol-level guarantee of recovery	Informational	Acknowledged
14	The v5 binary cannot replay pre-upgrade blocks because the power reduction is set unconditionally	Informational	Acknowledged
15	An error-acknowledgement refund finalizes with spendable legacy uzig when the faucet is short	Informational	Acknowledged
16	Active governance proposals with embedded uzig amounts are not rescaled	Informational	Acknowledged
17	Code quality improvements	Informational	Resolved
18	Wasm authorization message filters are not redenominated	Informational	Resolved

Detailed Findings

1. Depositing a trivial amount into the migration faucet or burner address ahead of the upgrade height trips the supply-conservation check and crashes every validator

Severity: Critical

- [app/upgrades/v5/bank.go:60-69](#)
- [app/upgrades/v5/bank.go:84](#)
- [app/upgrades/v5/faucet.go:24-38](#)
- [zutils/constants/v5migration.go:23-29](#)

The migrateBank's function per-holder sweep (`app/upgrades/v5/bank.go`) walks every address returned by `IterateAllBalances` and converts its `uzig` to `azig`, with exactly one exclusion: membership in the escrow set (`escrow.Contains(addr)`). Nothing else is filtered out.

The two accounts that back the post-upgrade `uzig`-returning mechanism, `v5migration_faucet` and `v5migration_burner` (`zutils/constants/v5migration.go`), do not exist as recognized module accounts on the currently-running pre-v5 binary; they are new names introduced by this very upgrade. Until the upgrade actually runs, `authtypes.NewModuleAddress("v5migration_faucet")` and its burner counterpart resolve to perfectly ordinary addresses that nothing on-chain blocks, protects, or even acknowledges, consequently, they are computable by anyone off-chain today, before the code is even deployed. An ordinary `MsgSend` to either address, before the upgrade runs, auto-creates a plain `BaseAccount` there, exactly as sending coins to any other previously-unused address would.

That is fatal, not merely a bookkeeping mismatch to be caught later. Every module-routed transfer in this handler resolves its module recipient through `AccountKeeper.GetModuleAccount`, which reads whatever account already exists at the derived address and, if one exists and is not a `ModuleAccountI`, panics outright (`panic("account is not a module account")`) in the vendored `cosmos-sdk x/auth/keeper`) rather than returning an error. `migrateBank`'s burner routing (`app/upgrades/v5/bank.go:84`, `SendCoinsFromAccountToModule(ctx, h.addr, MigrationBurnerName, uzigCoins)`) hits this resolution for every holder processed by the sweep, so if the burner address has been squatted, the panic fires on the very first holder the loop reaches, well before the sweep can even finish. If only the faucet address was squatted instead, the same panic fires slightly later, inside `preMintFaucet`'s module-to-module delivery (`app/upgrades/v5/faucet.go:24-38`). Either way, the supply-conservation check (`assertBankSupply`) is never even reached; the handler is gone before that point.

This is an unrecovered Go panic, not a returned error, so it does not get the benefit of the handler's usual fail-closed error path. It propagates out through `PreBlocker` into `FinalizeBlock` the same way any other fatal condition in this sequence does, and CometBFT treats that as

unrecoverable: every validator's process halts simultaneously at the upgrade height.

Reaching this requires nothing beyond an ordinary `MsgSend` for a dust amount of `uzig`, sent to either publicly-derivable address at any point before the upgrade height.

Recommendation

We recommend resolving both the faucet and burner module accounts once, explicitly, at the very start of the handler, and failing closed with a clean error if a non-module account already occupies either address, rather than relying on the SDK's default module-account resolution inside the sweep to panic. If a pre-existing non-module account is detected, its balance should be folded into the migration deliberately (moved into the sweep before the module account is created) rather than left to trigger an unrecovered panic partway through the handler.

Status: Resolved

2. Chain-wide denial of service enabled by an ordinary active governance proposal that becomes undecodable at the v5 upgrade height

Severity: Critical

- [app/upgrades/v5/gov.go:74-98](#)
- [app/upgrades/v5/errors.go:49-56](#)
- [x/dex/types/params.pb.go:36](#)
- [x/dex/types/message_update_params.go:11-21](#)
- [x/factory/types/params.pb.go:36](#)
- [app/v5_gov_undecodable_proposal_test.go:99-117](#)

The v5 upgrade handler's governance migration, `migrateGov` (`app/upgrades/v5/gov.go:74-98`), raw-decodes proposals from the KV-store instead of using the SDK's `gk.Proposals.Walk` iterator, specifically to avoid halting the upgrade on historical proposals whose embedded Any message can no longer be unpacked once the proto definitions change. For each stored proposal it then re-reads the record through `gk.Proposals.Get`, which does unpack the embedded message using the v5 binary's (post-upgrade) type definitions. If that unpack fails:

- A terminal proposal (passed/rejected/failed) is pruned from the store, no halt.
- An active proposal (deposit period or voting period) causes `migrateGov` to return `ErrActiveProposalUndecodable` (`app/upgrades/v5/errors.go:49-56`), which aborts the entire atomic v5 upgrade handler with no recovery path.

This diff also widens `x/dex's Params.CreationFee` and `x/factory's Params.CreateFeeAmount` from a protobuf `uint32` (wire type 0, varint) to a `math.Int` custom type (wire type 2, length-delimited). Any proposal embedding a `dex.MsgUpdateParams` or `factory.MsgUpdateParams` submitted before this change is stored with the field in the old wire format; decoding it under the new type definition fails with a wire-type mismatch.

An ordinary token holder can submit a completely standard dex or factory parameter-change governance proposal on the currently-live (pre-v5) binary, where `CreationFee/CreateFeeAmount` is still `uint32`. Standard proposal submission (`x/gov SubmitProposal`) only validates the embedded message against the binary running at submission time - signer check, `ValidateBasic()`, and message routability, all of which a genuine, unmodified param-change proposal satisfies trivially (`x/dex/types/message_update_params.go:11-21` only range-checks values and validates the authority address). No malformed bytes or special crafting are required. If that proposal is still in its deposit or voting period when the v5 upgrade height is reached, a height that is public knowledge well in advance, since it is announced via the `MsgSoftwareUpgrade` proposal that schedules the upgrade, every validator deterministically hits `ErrActiveProposalUndecodable` and the coordinated chain upgrade fails to apply.

Recommendation

We recommend ensuring the v5 gov migration can complete for an active proposal whose only decode failure is the known `CreationFee/CreateFeeAmount` wire-format widening, for example by attempting the same legacy-format hand-decode already used elsewhere in this upgrade for that field before falling back to `ErrActiveProposalUndecodable`, so that a legitimately submitted, in-flight proposal cannot unilaterally block the coordinated chain upgrade. Alternatively, we recommend extending the same reject-and-refund handling already applied to terminal proposals to active undecodable proposals as well, consistent with the team's own stated intent for this case.

Status: Resolved

3. A dust uzig deposit at a not-yet-created pool address halts the upgrade once that pool is created, because the DEX mirror reconciliation tolerates no divergence

Severity: Critical

- [app/upgrades/v5/dex.go:81-88](#)
- [x/dex/types/pool_address.go:14-16](#)
- [x/dex/keeper/msg_server_create_pool.go:200-210](#)
- [x/dex/keeper/send_restriction.go:41-46](#)

The `migrateDex` function closes out its rescaling of each pool by comparing what it computed the pool's native leg should be against what the pool address's real bank balance actually holds (`app/upgrades/v5/dex.go:81-88`), using `exact math.Int` equality with no tolerance. Any mismatch returns `ErrDexMirrorMismatch`, which is fatal to the whole atomic handler, the same as every other hard-stop in this sequence.

The address a pool will eventually occupy is knowable before the pool exists: `GetPoolAddress` (`x/dex/types/pool_address.go:14-16`) derives it purely from the next pool ID, a value visible in public chain state, with no dependency on the pool actually being created yet. `PoolSendRestriction` (`x/dex/keeper/send_restriction.go:41-46`) only starts blocking transfers to that address once `IsPoolAddress` recognizes it as a real, registered pool, which requires the pool to already exist, so an address reserved for a pool that hasn't been created yet accepts an ordinary `MsgSend` from anyone, same as any other unclaimed account.

When a pool is eventually created there, `CreatePool`'s pre-funding sweep only clears the two denoms that pool is actually being created with (`x/dex/keeper/msg_server_create_pool.go:200-210`, explicitly scoped this way to avoid a full balance scan), and the pool's stored `Coins` mirror is set to exactly those two amounts. Any third denom sitting at that address from before creation is left alone by both the sweep and the mirror. If that third denom happens to be `uzig`, the generic bank sweep run earlier in the upgrade treats the pool address like any other non-escrow holder, converting it to `azig` and delivering it right back to the pool, since nothing about the sweep excludes pool addresses from that pass. The pool's real balance now includes an `azig` amount its `Coins` mirror never recorded, and `migrateDex`'s strict equality check has nothing to do but fail.

Creating this condition costs one dust `MsgSend` to a predictable, currently-unclaimed address, at any point before the upgrade, followed by an ordinary `CreatePool` call for any pair that doesn't include the native denom, permissionless and requiring no special timing. This is the same actor model and blast radius as the faucet/burner pre-funding finding filed separately in this report,

reached through a different predictable address and a different exact check, but converging on the same outcome: the coordinated upgrade cannot complete until an operator manually identifies and resolves the specific pool.

Recommendation

We recommend having `CreatePool`'s pre-funding sweep account for every denom present at the pool address, not only the two it is being created with, so no balance can survive uncounted into the pool's mirror. Independently, we recommend having `migrateDex` reconcile an unexpected excess by folding it into the mirror or sweeping it elsewhere rather than aborting the entire upgrade over a single pool's divergence.

Status: Resolved

4. A wasm `StoreCodeAuthorization` grant with an attacker-chosen `CodeHash` can cause the authz migration's fail-closed substring scan and halt the upgrade

Severity: Critical

- [app/upgrades/v5/authz.go:116-171](#)
- [app/upgrades/v5/authz.go:212-217](#)
- [app/upgrades/v5/upgrades.go:123](#)

The `rescaleAuthorization` function handles a fixed set of `authz.Authorization` implementations explicitly: `SendAuthorization`, `StakeAuthorization`, `TransferAuthorization`, `GenericAuthorization`, and two wasm types, `ContractExecutionAuthorization` and `ContractMigrationAuthorization`. Anything else falls into the default branch, which runs a raw `bytes.Contains(anyBytes, []byte("uzig"))` scan and returns `ErrUnknownCoinBearingGrant` on any match, a fatal error that propagates through `migrateAuthz` and `app/upgrades/v5/upgrades.go:123` out of the atomic upgrade handler.

The wasm module actually registers a third `authz.Authorization` implementation, `StoreCodeAuthorization`, and it is not one of the cases in `rescaleAuthorization`'s switch. It therefore falls straight into the default branch and is subject to the substring scan.

`StoreCodeAuthorization` wraps a list of `CodeGrant` entries, each carrying a `CodeHash []byte` field that identifies which uploaded code the grant permits. That field is entirely attacker-chosen: the only validation applied is that it is non-empty and not a duplicate within the same grant list,

not that it is a genuine hash of anything. Nothing stops a grantor from setting CodeHash to any byte sequence they like, including one that happens to contain the ASCII bytes `uzig` as a substring.

Creating such a grant is completely permissionless: `MsgGrant` has no governance gate, no allowlist, and no code-level check that would reject a `StoreCodeAuthorization` whose `CodeHash` contains that substring. The grant carries no coins and does not touch supply, so every other step of the migration completes normally and the handler reaches `migrateAuthz`, where the byte scan fires and the upgrade aborts.

Recommendation

We recommend adding an explicit case for `StoreCodeAuthorization` in `rescaleAuthorization` that treats it as a no-op, since it carries no coin amount and never legitimately needs rescaling. More generally, we recommend replacing the content-blind substring scan in `anyBytesReferenceLegacyDenom` with a structural check against the known coin-bearing fields of each handled type, so that an opaque, attacker-influenced byte field on any unhandled type can never by itself halt the migration.

Status: Resolved

5. A group proposal embedding a wire-incompatible message permanently wedges the x/group end-blocker's chain-wide tally and pruning scan after the upgrade

Severity: Major

- [x/group/module/abci.go](#)
- [x/group/keeper/keeper.go](#)
- [x/group/keeper/genesis.go](#)
- [app/updates/v5/updates.go](#)

`x/group`'s end blocker runs `TallyProposalsAtVPEnd` and `PruneProposals` unconditionally every block. Both functions load their working set through a single chain-wide index scan keyed by voting-period-end time, and both wrap that scan with a bare `if err != nil { return nil }`: any decode error anywhere in the scan silently discards the entire due list for that call, not just the offending row.

Loading a stored `Proposal` from the group module's ORM decodes it through the standard codec path, which implicitly unpacks every embedded `Any` in its `Messages` field against the interface registry's *current* type definitions, the same way `x/gov`'s proposal store does. This app's own v5

diff widens `dex.MsgUpdateParams.CreationFee` (and the equivalent factory field) from a wire-incompatible `uint32` to `math.Int`, exactly the kind of change that already causes a decode failure for `x/gov`. `x/group` has no analogous migration step anywhere in the v5 upgrade handler, no `migrateGroup` call, and its own consensus version is untouched by this upgrade, so nothing ever attempts to reconcile a group proposal's stored bytes against the new type definitions before the end blocker starts reading them for real.

Creating a proposal that embeds such a message, through a group and policy that are themselves ordinary and permissionless to create, costs only gas and no deposit; a policy's voting period can legally be set to effectively zero, so the poisoned proposal becomes due for tally as soon as the first post-upgrade block runs. From that point, every chain-wide tally and prune scan hits the same decode error and returns early, meaning no group proposal anywhere on the chain, not just the poisoned one, is ever tallied or pruned again. Targeted deletion of the offending row doesn't help either, since removing it first requires reading it through the same decoding path. Exporting genesis hits the identical decode error while reading the proposal table and panics outright.

No funds are at risk and the chain itself keeps producing blocks; the effect is a silent, permanent disabling of the group module's automatic finalization and pruning, with proposal state accumulating unboundedly, recoverable only by a further coordinated software upgrade that adds the missing migration step.

Recommendation

We recommend adding a group-module migration step to the v5 upgrade handler that mirrors the guard already built for `x/gov`: walk stored group proposals using a decode path that tolerates an unpackable embedded message, and prune or otherwise neutralize any proposal that would poison the end blocker's scan, before the end blocker ever runs against post-upgrade state.

Alternatively, we recommend to consider removing the `x/group` module since it is deprecated.

Status: Resolved

6. CosmWasm contracts that track uzig amounts in their own internal storage will desync from their real balance after the v5 redenomination

Severity: Minor

- [app/upgrades/v5/bank.go:60-120](#)
- [wasmbinding/message_plugin.go:181-861](#)
- [wasmbinding/query_plugin.go:65-260](#)

The v5 upgrade handler's bank sweep (`migrateBank`, `app/upgrades/v5/bank.go:49-128`) walks every non-escrow holder via `IterateAllBalances` and rescales its uzig balance to azig uniformly. The bank keeper stores balances purely by `sdk.AccAddress`, with no concept of "this address belongs to a CosmWasm contract", a contract instance address is bech32, indistinguishable from any other account, so its raw bank balance is correctly rescaled to `1e12` like any other holder's (confirmed no `wasm-storage-aware` special-casing exists on this path).

The gap is with what the sweep cannot see: `x/wasm` gives each contract instance its own isolated KV storage namespace, opaque to the chain outside of the contract's own entry points. The v5 handler's 19 migration steps touch only bank balances/supply and the native module stores (`auth`, `staking`, `distribution`, `mint`, `gov`, `authz`, `feegrant`, `ratelimit`, `tokenwrapper`, `dex`, `factory`, `escrow`); none of them open or write to contract-owned storage, and there is no generic, safe way for a chain-level upgrade handler to do so, since it cannot know what a given `int128` in a contract's storage represents (a uzig amount, an unrelated counter, a block height, etc.).

Concretely: a CosmWasm contract that tracks its own internal accounting value denominated in uzig - for example a `total_reserve`, `exchange_rate`, or `collateral_amount` field in its own state, separate from its actual bank balance - will, after the upgrade, hold `1e12×` more azig in its real balance than its internal ledger still believes it holds. Any contract logic that computes results from that internal value (a share-redemption ratio, a collateralization check, an AMM reserve calculation) will be off by the same factor.

This risk is confined to contracts that maintain independent internal accounting; a contract that relies entirely on live bank-balance queries, or that is built on this chain's native `x/dex/x/factory` bindings (`wasmbinding/message_plugin.go`, routed through those modules' own msg servers), is unaffected, since that state lives in module stores the v5 handler does correctly rescale.

Recommendation

We recommend publishing an upgrade-integrator notice alongside the existing wallet/explorer/exchange guidance, stating that a CosmWasm contract's bank balance is rescaled automatically but any internal uzig-denominated value in its own storage is not, and

must be migrated by the contract itself (e.g. via a Migrate entry point) if the contract needs its internal ledger to stay consistent with its real balance.

Status: Acknowledged

The client acknowledges the issue stating that contract-owned KV storage is intentionally opaque to the chain upgrade handler, as any integer field could represent a native amount, a counter, a block height, or a price, making safe chain-level rewriting impossible. Remediation is therefore the responsibility of individual contract authors, and the client has treated this finding as a disclosure rather than a code change. A developer notice has been written and distributed to all known contract teams, covering which state is migrated automatically, which requires manual action, and a recommended migration procedure including rehearsal on the test network ahead of the upgrade height.

7. Near-zero DEX pool-creation and factory denom-creation fees on freshly initialized chains due to unrescaled default fee constants

Severity: Minor

- [x/dex/types/params.go:40-44](#)
- [x/factory/types/params.go:16-19](#)
- [app/basic_manager_overrides.go:1-64](#)

The v5 upgrade handler correctly rescales the *persisted* `CreationFee` (`x/dex`) and `CreateFeeAmount` (`x/factory`) parameters by $\times 1e12$ for any chain that upgrades from a pre-v5 state. The *default* constants used to build `DefaultParams()` - and therefore `DefaultGenesis()` for any chain that initializes fresh rather than upgrading - were left at their pre-redenomination values:

- `x/dex/types/params.go:44` - `DefaultCreationFee = math.NewInt(100 * 1_000_000)` (100 ZIG at the legacy 6-decimal scale).
- `x/factory/types/params.go:19` - `DefaultCreateFeeAmount = math.NewInt(1000)` (same pattern).

This repository already has a mechanism for exactly this class of problem: `app/basic_manager_overrides.go`'s `OverrideDenomGenesisDefaults` wraps `x/staking`, `x/mint`, and `x/gov`'s `DefaultGenesis()` so that `zigchaind init` produces zigchain-correct (azig) defaults instead of the SDK's generic placeholder denom. `x/dex` and `x/factory` are not included in this override, so their `DefaultGenesis()` is never adjusted for the new 18-decimal scale.

A chain or testnet initialized fresh from this codebase post-v5 (via `zigchaind init`, not an upgrade) therefore gets a pool-creation fee of `100_000_000` base units ($\approx 1e-10$ ZIG at 18 decimals) and a denom-creation fee of `1000` base units ($\approx 1e-15$ ZIG) - both effectively free. This removes the intended spam-resistance friction on DEX pool creation and factory denom creation for any new network launched from this code. It does not affect the already-upgraded mainnet path, where the persisted values are correctly rescaled by the upgrade handler.

Recommendation

We recommend rescaling `DefaultCreationFee` and `DefaultCreateFeeAmount` by the same $\times 1e12$ factor the upgrade handler applies to the persisted values, or adding `x/dex` and `x/factory` to `OverrideDenomGenesisDefaults` following the existing `staking/mint/gov` pattern.

Status: Resolved

8. Node minimum gas prices are not migrated and require validator coordination

Severity: Minor

- [cmd/zigchaind/cmd/config.go:43](#)
- [app/ante.go:69](#)

The minimum gas price is per-validator node configuration in `app.toml`, not on-chain state, so the upgrade handler has no object to migrate. The compiled default is commented out (`cmd/zigchaind/cmd/config.go:43`), the chain wires no global-fee or fee-market module, and the ante handler falls back to the node-local minimum gas price during `CheckTx` (`app/ante.go:69`). After the redenomination, each validator must update the configured minimum gas price from the `uzig` figure to the `azig` equivalent.

If a validator retains a `uzig`-denominated minimum gas price, the node rejects normal traffic from its mempool. If a validator performs only a denom swap (`uzig` $\rightarrow$ `azig`, keeping the numeric value), the effective price is 10^{12} times cheaper, exposing the node to a spam/denial-of-service surface. The condition is node-local, non-consensus, and recoverable by editing `app.toml` and restarting.

Recommendation

We recommend documenting a mandatory runbook step instructing every validator to set the minimum gas price in azig and to multiply the numeric price by 10^{12} . Emphasize both halves, because the denom-only swap is the dangerous partial fix.

Status: Acknowledged

The client acknowledges the issue stating that the recommended mitigation is already addressed in the validator upgrade runbook, which falls outside the audited code scope. The runbook explicitly covers both required steps: updating the denomination from uzig to azig and multiplying the numeric value by 10^{12} , with the denom-only swap flagged as a dangerous partial fix. The client considers the risk adequately managed through this existing operational documentation.

9. DEX AddLiquidity/RemoveLiquidity response fields mislabel Base/Quote by sorted array position instead of caller-supplied role after the v5 redenomination

Severity: Minor

- [x/dex/keeper/msg_server_add_liquidity.go:51-92](#)
- [x/dex/keeper/msg_server_add_liquidity.go:241-315](#)
- [x/dex/keeper/msg_server_remove_liquidity.go:183-194](#)
- [proto/zigchain/dex/tx.proto:151-172](#)
- [app/upgrades/v5/dex.go:49-68](#)

Pool.Coins is an sdk.Coins-shaped slice, which the Cosmos SDK always keeps sorted by denom string. Before the v5 redenomination, the native denom uzig sorted *after* factory subdenoms (coin.<addr>.*); after the redenomination the native denom is azig, which sorts *before* them. app/upgrades/v5/dex.go correctly handles this for on-chain storage: it re-sorts Pool.Coins and rekeys the pool-uniqueness index after the rename.

The response messages for AddLiquidity and RemoveLiquidity do not track this correctly. In AddLiquidity (x/dex/keeper/msg_server_add_liquidity.go:51-92), the handler sorts the caller's (msg.Base, msg.Quote) alphabetically into sortedCoins, verifies only that the set of denoms matches the pool (not which one the caller called Base vs Quote), and calls CalculateLiquidityShares(pool, params, sortedCoins[0], sortedCoins[1]). The resulting MsgAddLiquidityResponse.ActualBase/ActualQuote therefore reflect "whichever denom sorts first/second," not "whichever denom the caller labeled Base/Quote." RemoveLiquidity

(x/dex/keeper/msg_server/remove_liquidity.go:183-194) is fully positional: `MsgRemoveLiquidityResponse{Base: coinsOut[0], Quote: coinsOut[1]}`, with no caller-supplied label to compare against at all.

Because every `sdk.Coin` still carries its own `Denom` field, the underlying token accounting is not corrupted, a consumer that reads `.Denom` gets the correct amount and asset. The risk is that any `azig/factory-token` pool has its `base/quote/actual_base/actual_quote` field assignment flip at the v5 upgrade height (since `azig` now sorts first where `uzig` previously sorted second), with no on-chain error or signal that the mapping changed. An off-chain consumer (indexer, bot, UI, or a `CosmWasm` contract consuming these responses via `wasmbinding`) that trusts the field `name` rather than reading `.Denom` will silently attribute the wrong physical asset to that field for every pre-existing `azig`-paired pool.

Recommendation

We recommend deriving `ActualBase/ActualQuote` (and the `RemoveLiquidity` equivalents) by matching against the caller's original `msg.Base/msg.Quote` denom rather than by sorted array position, so the response field name always corresponds to what the caller requested regardless of the pool's internal storage order.

Status: Acknowledged

The client acknowledges the issue stating that the sorted-position semantics in the response fields are an intentional design decision, introduced to resolve a prior critical finding requiring pool reserves to be credited based on sorted token order. Relabelling response fields to match caller-supplied roles is not feasible consistently across both messages, as `RemoveLiquidity` carries no caller-supplied base/quote arguments to match against. The client has instead documented the convention in the proto definitions, requiring consumers to read `.Denom` directly rather than infer the asset from the field name, and has added a regression test to prevent future changes from silently reintroducing the original critical vulnerability.

10. Packet-forward-middleware forwarding of a returning uzig voucher always fails due to migrateuzig converting the intermediate balance to azig before the forward attempt

Severity: Minor

- [app/ibc.go:197-241](#)
- [x/migrateuzig/ibc_module.go:81-103](#)
- [x/migrateuzig/ibc_module.go:173-203](#)

The IBC receive-side middleware stack is composed, from outermost to innermost, as `RateLimit` → `PacketForwardMiddleware` (PFM) → `migrateuzig` → `TokenWrapper` → `Callbacks` → `Transfer` (`app/ibc.go:197-241`). PFM's `OnRecvPacket` runs first and, for a packet carrying a forward memo, invokes the inner stack to receive funds at an intermediate account before forwarding them onward.

`migrateuzig's OnRecvPacket` (`x/migrateuzig/ibc_module.go:81-103`) does not rewrite the packet's `FungibleTokenPacketData` before returning control to the caller: after crediting the intermediate account, it burns the received uzig and pays out azig from the faucet reserve in place, leaving the original packet data (still describing uzig) unchanged. When PFM then attempts to forward using that original packet data, it tries to send uzig out of an account that `migrateuzig` has already converted to hold only azig, and the forward fails with an insufficient-funds error.

Because IBC packet processing for a receive is wrapped in a single cache context that is only committed if the final acknowledgement is successful, this failure is fully atomic: the entire chain of operations, including `migrateuzig's` burn/mint, is rolled back, and an error acknowledgement is returned to the source chain for the standard sender refund. No funds are placed at risk. The practical effect is that any returning-uzig voucher carrying a packet-forward-middleware memo will always fail to complete its intended forward hop after the v5 upgrade.

Recommendation

We recommend documenting that packet-forward-middleware memos are not supported on returning uzig vouchers post-upgrade, and adding regression test coverage for this combination so the fail-safe behavior is verified rather than assumed.

Status: Acknowledged

The client acknowledges the issue stating that the behavior has been documented and verified through pinned regression tests rather than addressed with a code change. The failure mode is accepted as a known limitation of returning uzig vouchers carrying

packet-forward-middleware memos after the v5 upgrade. No funds are at risk due to the atomic rollback, and the client considers test coverage sufficient to confirm the fail-safe behavior going forward.

11. Staking HistoricalInfo retains old-scale validator tokens after the upgrade

Severity: Informational

- [app/upgrades/v5/staking.go:37](https://github.com/cosmos/cosmos-sdk/blob/master/app/upgrades/v5/staking.go#L37)

The `migrateStaking` function does not rescale `HistoricalInfo`, so stored `HistoricalInfo` entries retain old-scale validator Tokens after the upgrade.

`Validator.Tokens` is a bond-denom-only integer, so the stored value is merely frozen before the rescale, not corrupted. The staking end blocker rebuilds `HistoricalInfo` from live, already-rescaled validator state every block and prunes entries older than the `HistoricalEntries` window (default 10000), so all stale entries age out within that window. The only consumer is the staking gRPC query, which returns the value without arithmetic or comparison. No consumer malfunctions.

Recommendation

We recommend deleting all `HistoricalInfo` during the upgrade as defense-in-depth; it is rebuilt on the next block.

Status: Acknowledged

The client acknowledges the issue stating that the old-scale `HistoricalInfo` entries are historically accurate rather than corrupted, as validators genuinely held those token amounts at the pre-upgrade height and rescaling would fabricate incorrect values. The only consumer, the staking gRPC query, returns the value verbatim without performing arithmetic, and the IBC path that previously consumed historical validator sets was removed in `ibc-go v9`. Entries self-heal as the end blocker rebuilds and prunes `HistoricalInfo` within the `historical_entries` window, and adding a bulk deletion introduces unnecessary consensus-critical apphash surface for a no-impact finding.

12. An IBC destination callback executes against uzig while the receive settles in azig

Severity: Informational

- [x/migrateuzig/ibc_module.go:82](#)
- [app/ibc.go:214](#)
- [app/ibc.go:230](#)

On the v1 receive stack, the migrateuzig module wraps the callbacks middleware (app/ibc.go:230 is outer to app/ibc.go:214), and OnRecvPacket invokes the inner stack through im.app.OnRecvPacket before it performs the conversion (x/migrateuzig/ibc_module.go:82). When a returning native-uzig voucher names a contract as the receiver and carries a dest_callback memo, the wasm destination callback fires inside the inner stack, so the contract observes the uzig packet, and only afterward does convert burn the uzig and pay azig from the faucet.

Two outcomes arise. If the callback moves the credited uzig, the subsequent burn fails, OnRecvPacket returns an error acknowledgement, and the entire receive reverts atomically. If the callback does not move the uzig, the contract's callback logic executes against uzig while the settled balance the contract holds is azig.

The path where the callback moves the uzig is fail-closed, and on the other path value is conserved one-to-one at $\times 10^{12}$. The residual concern is that a contract that records, forwards, or prices the uzig denomination inside its destination callback becomes inconsistent with the final azig settlement. The condition is reachable only while native uzig can still return home after the upgrade and the receiver is a contract that both opts into destination callbacks and acts on the uzig denomination, and it is confined to the migration window.

Recommendation

We recommend documenting that returning native uzig converts after the destination callback runs, so contract authors do not rely on the received denomination inside the callback. Optionally, reject a dest-callback receive of returning uzig during the migration window.

Status: Acknowledged

The client acknowledges the issue stating that the behavior has been documented for contract authors and no code changes are planned. The client accepts the residual risk, which is confined to the migration window and only affects contracts that both opt into destination callbacks and act on the uzig denomination within them. As value is conserved one-to-one across the conversion, the client considers the documented guidance sufficient mitigation.

13. A long-running v5 upgrade handler cannot be aborted or forked by CometBFT consensus round timeouts, but slow validators risk downtime-jailing rather than a protocol-level guarantee of recovery

Severity: Informational

- [app/upgrades/v5/upgrades.go](#)
- [app/setup_handlers.go](#)

The v5 upgrade handler (`app/upgrades/v5/upgrades.go`) performs substantial state migration work inside `PreBlocker` at the coordinated upgrade height, iterating bank balances, governance proposals, staking/distribution records, `authz/feegrant` grants, DEX pools, and IBC escrow state. No application-level timeout bounds this work; it runs to completion however long that takes.

This finding documents, for completeness, that CometBFT's own consensus timeout parameters (`timeout_propose`, `timeout_prevote`, `timeout_precommit`, `timeout_commit`) do not create a competing failure mode here. `FinalizeBlock` (where the migration runs) executes only after a block has already been decided by BFT consensus ($+2/3$ precommits recorded) and durably written to the block store; the round timeouts govern only the propose/prevote/precommit voting phases that precede that decision, and cannot fire during or interrupt block execution. `FinalizeBlock/Commit` are themselves invoked with an unbounded context and no timeout of their own; `timeout_commit` is a fixed wait applied only after they complete, not a bound on their duration. When a validator running the pre-upgrade binary halts at the upgrade height (no registered handler for the "v5" plan) and is manually restarted with the upgraded binary, the restart replays the same already-decided block locally rather than triggering a new round of voting, so round timeouts are likewise irrelevant to that recovery path.

The practical consequence of an unusually long migration is therefore operational, not consensus-breaking: validators that finish later than others cause the chain to wait for $+2/3$ voting power to be ready before the next height can proceed (a liveness delay, not a fork), and any validator that takes long enough risks being jailed for downtime by the standard slashing module mechanism, independent of anything specific to this migration.

Recommendation

We recommend running the environment-gated snapshot benchmark (`TestV5SnapshotLiveUpgrade`) against a realistic production-scale state before the coordinated upgrade so operators have an actual timing expectation.

Status: Acknowledged

The client acknowledges the issue stating that the v5 upgrade handler has been tested against bootstrap snapshots and a full archive node, with migration completing in approximately two minutes in both cases. Based on these results, the client has updated its operator documentation to set an expected upper bound of five minutes for the upgrade process. The client considers the operational risk acceptable given the measured timing, which falls well within a range that avoids validator downtime-jailing under normal circumstances.

14. The v5 binary cannot replay pre-upgrade blocks because the power reduction is set unconditionally

Severity: Informational

- [app/config.go:36](#)
- [app/config.go:34](#)
- [app/upgrades/v5/staking.go:19](#)

The `init` function in `app/config.go:36` sets the global `sdk.DefaultPowerReduction` to 10^{18} unconditionally, with no height guard. A height guard is not straightforward, because the value is a global package variable set at binary initialization, before any block or height context exists. Consensus power is computed as `tokens.Quo(powerReduction)`, an integer division that truncates. The v5 migration scales validator tokens $\times 10^{12}$, so on post-upgrade (18-decimal) state the power is exactly invariant: $(t \times 10^{12}) / 10^{18} == t / 10^6$. This invariance holds only for post-upgrade state, which is the justification for not rebuilding the power index (`app/upgrades/v5/staking.go:19`).

When the v5 binary evaluates pre-upgrade (6-decimal) token state, that is, when it replays or serves a pre-upgrade block, it computes `oldTokens / 10^{18}`, which truncates to zero for every realistic validator. A validator would need at least 10^{18} `uzig` (10^{12} ZIG) bonded to earn even a single unit of power, which is orders of magnitude beyond any pre-upgrade holding. The entire validator set therefore collapses to zero power, and the real validator-set-update path (`ApplyAndReturnValidatorSetUpdates`) emits power-zero updates, diverging from the canonical app hash.

Recommendation

We recommend documenting that the v5 binary is correct only for post-upgrade (18-decimal) state and must not replay or serve pre-upgrade blocks. To bootstrap a node, either run `cosmovisor` with the v4 binary in `cosmovisor/genesis/bin` and the v5 binary in

cosmovisor/updates/<name>/bin, so pre-upgrade heights execute under v4, or state-sync from a post-upgrade snapshot. Do not run the v5 binary standalone from genesis.

Status: Acknowledged

The client acknowledges the issue stating that the unconditional power reduction setting is intentional design, paired with the upgrade handler's token rescaling to keep consensus power exactly invariant across the migration. The cosmovisor deployment contract ensures the v5 binary is never executed against pre-upgrade state, making a height guard both infeasible and unnecessary. Existing documentation and the validator runbook already mandate the correct bootstrapping procedure, so no code change is required.

15. An error-acknowledgement refund finalizes with spendable legacy uzig when the faucet is short

Severity: Informational

- [x/migrateuzig/ibc_module.go:167](#)
- [x/migrateuzig/ibc_module.go:119](#)
- [x/migrateuzig/ibc_module.go:128](#)
- [x/migrateuzig/ibc_module.go:131](#)

In the migrateuzig IBC middleware, convertRefund converts a refunded uzig amount to azig from the migration faucet. When the conversion fails, convert returns an error at its fail-closed pre-check before any state change, convertRefund logs the error and returns nil in line 167. OnAcknowledgementPacket (x/migrateuzig/ibc_module.go:119) and OnTimeoutPacket (x/migrateuzig/ibc_module.go:128) both return this value, so IBC core finalizes the acknowledgement while the wrapped ICS-20 handler has already refunded plain uzig to the sender.

The sender retains spendable legacy uzig after the migration, reintroducing the legacy denom. No funds are lost, the value is preserved as uzig, and reachability is low, because the faucet is pre-minted to cover the entire outstanding uzig supply and every conversion burns uzig one-to-one. The behavior is an intentional, documented design choice (x/migrateuzig/ibc_module.go:131).

Recommendation

We recommend persisting a permissionless conversion claim that the sender can redeem later, rather than silently leaving `uzig`, so the no-legacy-denom invariant holds even when the faucet is transiently short. Propagating the error is not advised on an ordered channel, as it would stall the channel.

Status: Acknowledged

The client acknowledges the issue stating that the faucet shortage scenario is considered unreachable in practice, as the pre-minted supply is calculated to cover all outstanding legacy tokens and the total supply decreases monotonically through IBC claims with no refill path. The client further argues that retaining the refund as legacy tokens is the correct defensive behavior, since it avoids creating an unbacked liability against the faucet. The recommended claim mechanism is rejected because a permanent shortage would render any such claim equally unredeemable, and affected holders retain fully spendable tokens convertible through any available channel.

16. Active governance proposals with embedded `uzig` amounts are not rescaled

Severity: Informational

- [app/upgrades/v5/gov.go:34](#)
- [app/upgrades/v5/gov.go:50](#)
- [app/upgrades/v5/gov.go:88](#)

The `migrateGov` function rescales deposit parameters, proposal total deposits, and in-flight deposit amounts, but deliberately leaves the embedded `Messages` of a proposal opaque (`app/upgrades/v5/gov.go:50`), because decoding them would risk halting on historically unpackable messages. An embedded `uzig` coin therefore retains its old scale and denom.

Consequently, a proposal that embeds a `uzig` amount and passes after the upgrade executes with a stale amount or denom. Most such messages fail at execution, and the stock gov end blocker marks the proposal failed in a cached branch without halting the chain; a parameter-only message may succeed with a stale denom, which is recoverable through governance. Active undecodable proposals are already rejected fail-closed (`app/upgrades/v5/gov.go:88`).

Recommendation

We recommend documenting operational practice of voting down active proposals before the upgrade, and document the residual behavior for proposals submitted close to the upgrade height.

Status: Acknowledged

The client acknowledges the issue stating that the informational classification is appropriate and that no code change is warranted. The team agrees the risk is acceptable given that most affected proposals would fail at execution without halting the chain, and any edge cases involving stale parameter messages are recoverable through governance. The client accepts a documentation-only resolution and will rely on operational practices to manage active proposals around the upgrade height.

17. Code quality improvements

Severity: Informational

- [app/upgrades/v5/bank.go:127](#)
- [app/upgrades/v5/bank.go:170](#)
- [app/upgrades/v5/redenom/redenom.go:144](#)
- [x/tokenwrapper/keeper/keeper.go:465](#)
- [x/tokenwrapper/module/handlers.go:40](#)

The following are minor code quality observations that do not pose security risks but could be addressed for improved maintainability:

1. In `app/upgrades/v5/bank.go:127`, the `migrateBank` function returns `escrow.TotalUzigs` as a redundant return value since the caller already has access to this value from the earlier context in `app/upgrades/v5/upgrades.go:80`.
2. In `app/upgrades/v5/bank.go:170`, the `reconcileLegacyMetadata` function returns silently if legacy denom metadata is not found. Consider returning an error to force investigation if the metadata is missing, as it should always exist during a valid upgrade.
3. In `app/upgrades/v5/redenom/redenom.go:144`, the `ScaleDecCoins` function uses inline multiplication `c.Amount.MulInt(ScaleFactor)` instead of an existing `ScaleDec` helper function, which could improve code consistency.
4. In `x/tokenwrapper/keeper/keeper.go:465`, the `ScaleDownTokenPrecision` function comment states "ScaleDownTokenPrecision scales down the token precision from 18 to 6 decimals". This is incorrect, as the migration has updated the scaling to be 0 decimals. The same issue occurs in `x/tokenwrapper/module/handlers.go:40` of the `handleRefund` function.

Recommendation

We recommend implementing the following recommendations:

1. Remove the redundant `escrow.TotalUzig` return value from `migrateBank`.
2. Consider returning an error in `reconcileLegacyMetadata` if legacy denom metadata is not found so developers can investigate missing metadata.
3. Use the existing `ScaleDec` helper function in `ScaleDecCoins`.
4. Update comments in the `ScaleDownTokenPrecision` and `handleRefund` functions to accurately reflect the decimal conversion direction.

Status: Resolved

18. Wasm authorization message filters are not redenominated

Severity: Informational

- [app/upgrades/v5/authz.go:156](https://github.com/cosmos/cosmos-sdk/issues/156)
- [app/upgrades/v5/authz.go:181](https://github.com/cosmos/cosmos-sdk/issues/181)

The `rescaleWasmContractGrants` function rescales only the `fundLimit` of a wasm contract grant and never touches the grant's `Filter`. The `AcceptedMessagesFilter` raw message bytes are carried through the migration unchanged. Unlike the unhandled types in the `StoreCodeAuthorization` finding, the wasm execution and migration authorization types have an explicit case, so control never reaches the fail-closed default branch, and the migration does not halt.

If a contract's accepted message body embeds a denom string such as `uzig`, the stored filter continues to match the stale `uzig` payload and rejects the `azig` payload the grantee would send after the upgrade, so the grant silently stops working. Only the grantee is affected, and the condition is fully recoverable by revoking and re-granting. Rewriting the opaque payload generically is infeasible without the contract schema.

Recommendation

We recommend documenting explicitly that wasm authorization message filters are intentionally not redenominated, so grantors can re-issue affected grants after the upgrade.

Status: Resolved